

Real-time Firmware Design for Collaborative Robots



Gawtam C R (ED19B007)
Department of Engineering Design, IIT Madras

Jagannath Raju (Mentor)
CTO, Systemantics India Pvt. Ltd

Introduction

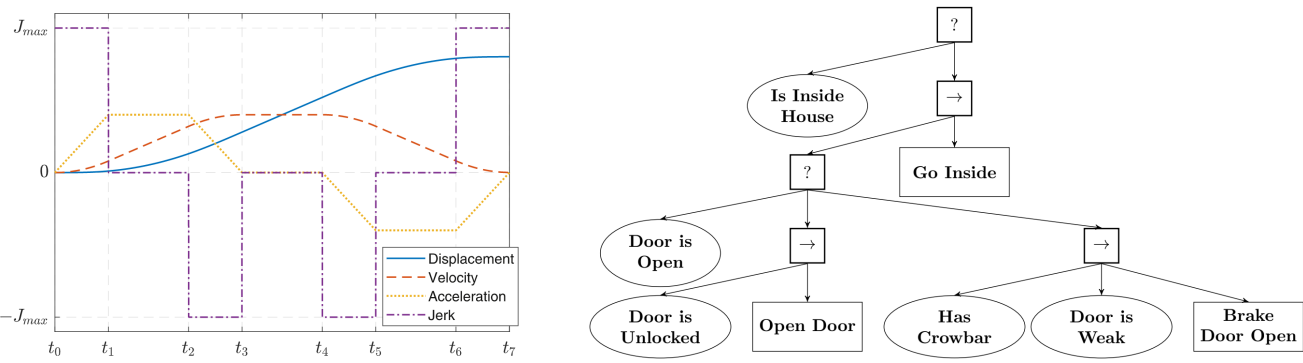
- A collaborative robot, or cobot, is specifically designed to work alongside human operators, facilitating a seamless collaboration that enhances productivity, efficiency, and safety in various industries. (especially Academic, Surgical, and Laboratory environments). The key distinguishing features of collaborative robots are:
- Flexibility & Adaptability
 - Force/Torque Limiting.
 - Collision Avoidance, Kinesthetic Teaching
 - Workspace awareness

Objectives

- Implement trajectory generation of a 6-DOF serial manipulator.
- Build a system controller with real-time POSIX features in Linux (RaspberryPi)
- Incorporate a robust design architecture using Domain Neutral Abstraction and UML Diagrams.
- Integrate ROS2 (Moveit2, Gazebo) and Behaviour Tree based learning feature integrated into the company product portfolio of cobots

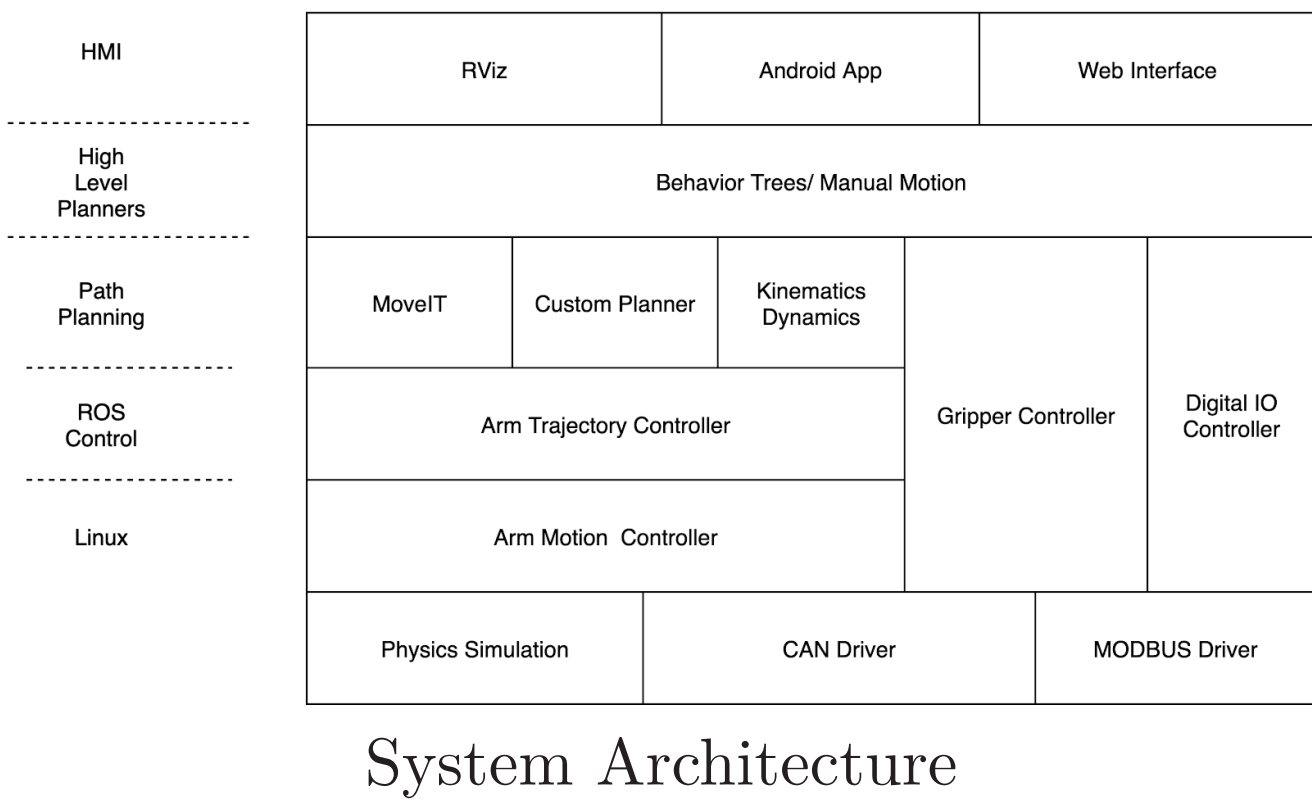
Methodology

- Trajectory Generation: The Trajectory is split into 7 zones according to S-Curve, which reduces mechanical shock by accelerating & decelerating the motor slowly.



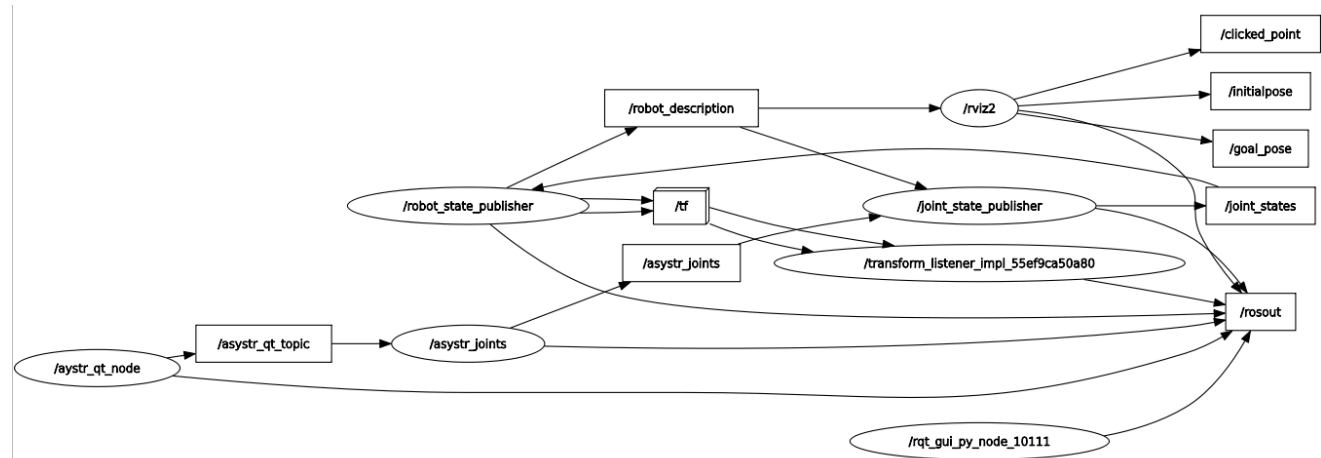
S-Curve & Behavior Tree with backchaining

- Behavior tree is a high-level abstraction for task planning. It is modular, hierarchical, readable, and reusable and guarantees convergence to the goal



Results

- Built a complete and robust system controller for the company’s portfolio of cobots from scratch.
- Implemented behavior trees for task planning of the robotic arm for applications like laboratory pipetting, fast-food kiosks, and inspection of parts.
- Integrated ROS2 with the company’s cobots. Created URDFs and SRDFs, implemented Hybrid Motion Planning using Moveit2, and simulated the dynamics of the cobot in Gazebo.



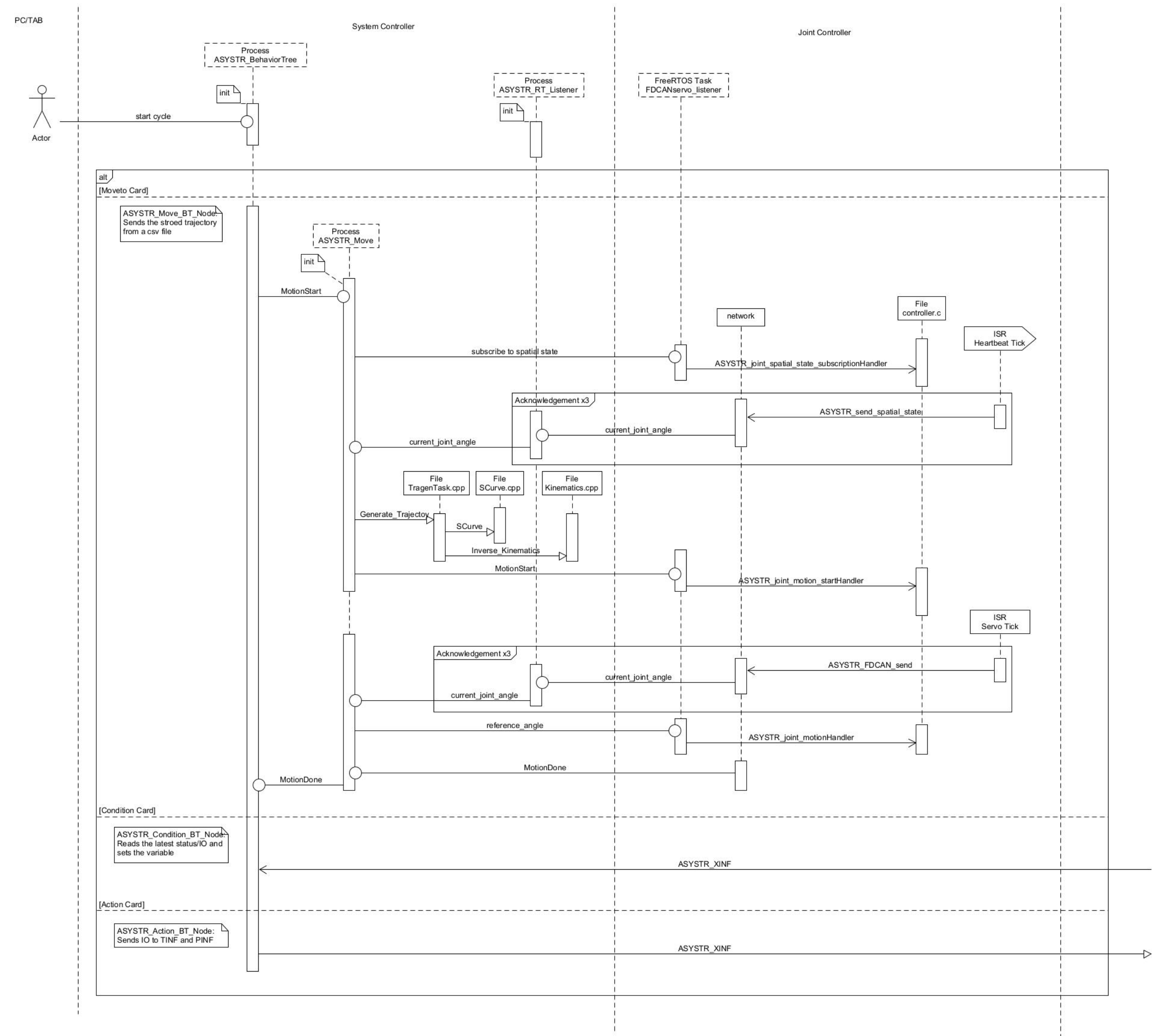
Graph of ROS2 nodes and topics

Learning Outcomes

- Kinematics & dynamics of serial robots
- Understood System Design, ROS2, Linux, and Real-time Operating systems.
- Daily Design team meets were helpful to learn on what others in the company were working on, like PCB Design, SPI/I2C, Motor PID Control, Force Torque Sensing, Depth and Image sensing, etc...

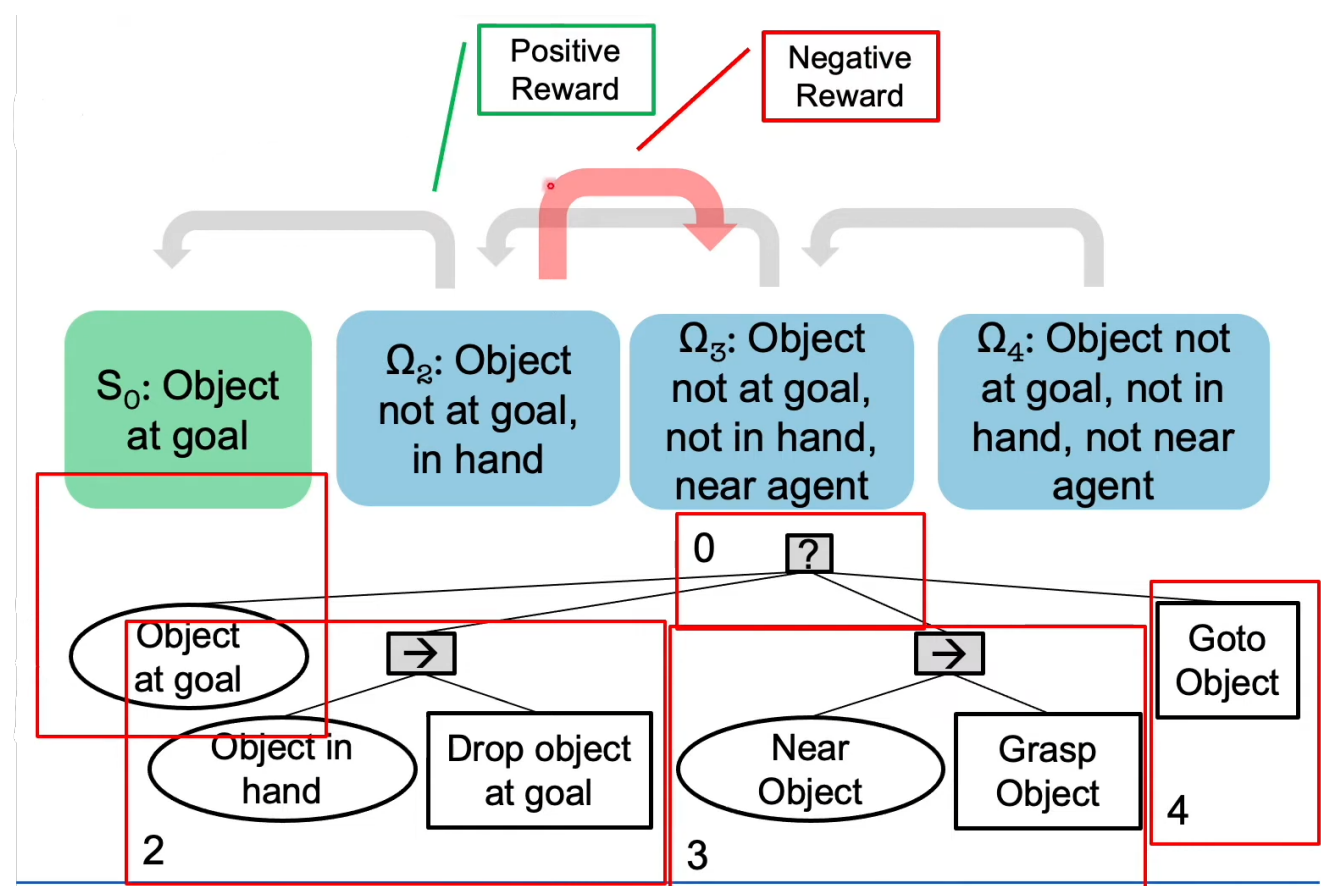
System Design

- Real-time processes are used in System controller to perform time-critical tasks which affect the robot’s motion. (using Linux PREEMPT RT patch)
- The below sequence diagram indicates the different Processes, threads, and APIs of the system controller and joint controller. The communication between processes and controllers is via Sockets (SocketCAN), FDCAN, semaphores and Signal Interrupts.



Future Scope

- Implement Blending in trajectory when more than two waypoints are given.
- Started to work on Disturbance observer for collision detection using impedance control, this can also be extended for Kinesthetic teaching (admittance control)
- Admittance control is the opposite, or dual of the ubiquitous method of impedance control. The naming reflects the causality of the used virtual model dynamics.
- Behaviour trees also lend themselves to reinforcement learning methods to refine and extend the behavior based on data collected using the robotic arm.)



References

Official Documentations of : ROS2, Systemantics, BehaviorTree.CPP, man7.org, POSIX and SocketCAN.